

Bridge Pattern for Shape Rendering

Assignment 3 | Software Design Patterns | ShP-2216
Astana IT University - School of Computer Engineering

Student Olzhabekov Ali **Group** SE-2515

1 Introduction

The project implements the suggested Shape-Renderer topic in Java. A circle and a square can be drawn as either an SVG vector image or a PNG raster image. The same shape object can change its rendering implementation at runtime while preserving its dimensions.

Bridge fits because the domain has two separate reasons to change: shape behavior and rendering technology. Shape classes describe what to draw; renderer classes decide how to encode the drawing. A Renderer reference inside Shape connects these sides through object composition. This avoids classes for every combination, such as VectorCircle and RasterCircle. With m shape types and n rendering technologies, the combination-specific subclass approach can require $m \times n$ variants; the bridge separates those variants into the two sides.

Pattern roles in the implementation

Role	Project element	Responsibility
Abstraction	Shape	Holds the Renderer reference and exposes draw and setRenderer.
Refined Abstractions	Circle and Square	Keep a radius or side and delegate drawing.
Implementor	Renderer	Declares drawCircle and drawSquare.
Concrete Implementors	VectorRenderer and RasterRenderer	Create SVG geometry or PNG pixels.
Client	Main	Chooses implementations and switches existing shapes.

Bridge and Adapter

Bridge is used here to separate two dimensions in the design. Adapter solves a different problem: making an existing incompatible API satisfy the interface a client expects. This project does not wrap a legacy drawing API to repair an incompatibility. Both renderers are deliberately written to the same Renderer contract. Their interface-based connection is the bridge.

2 UML and runtime composition

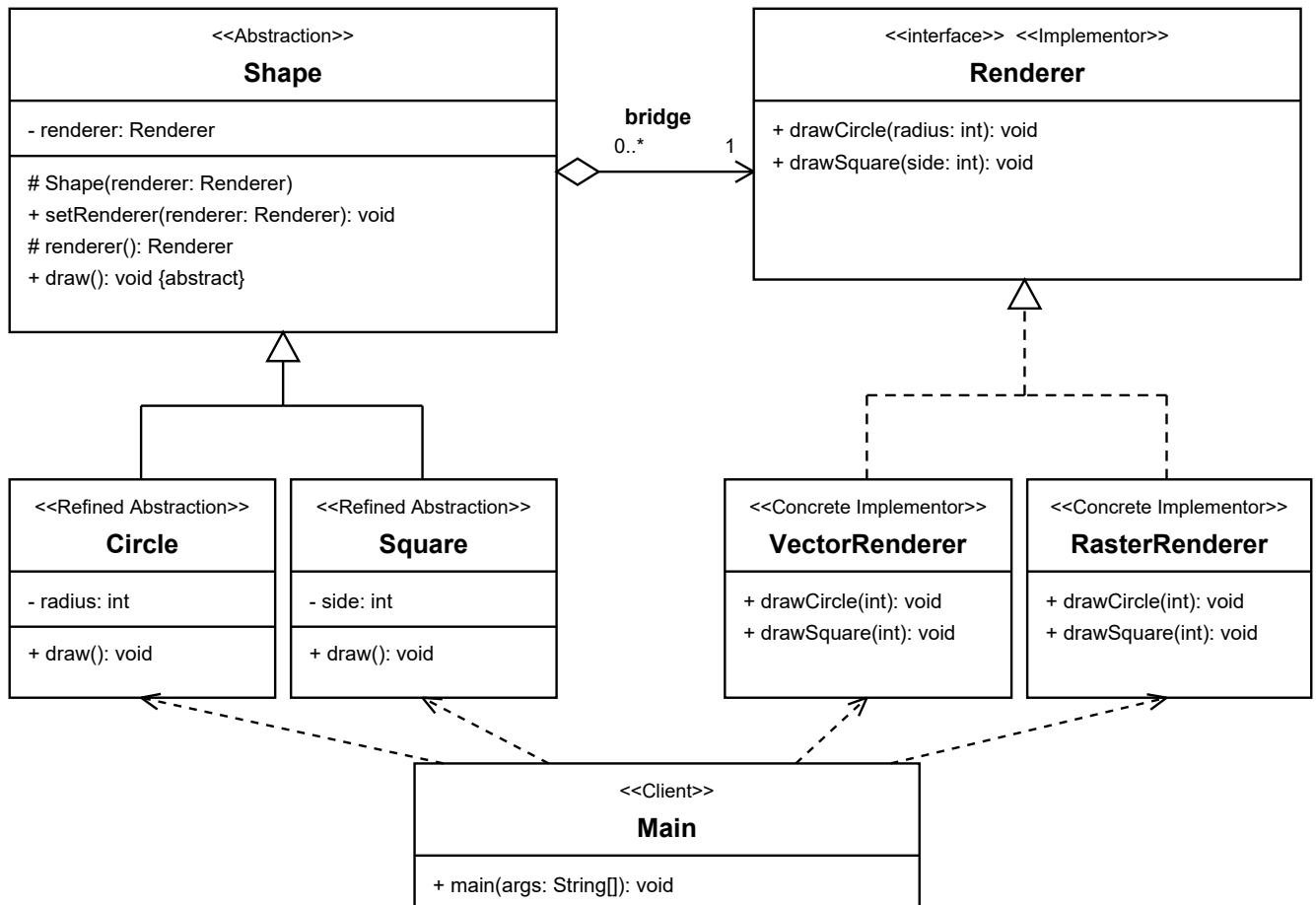


Figure 1. The shape hierarchy and the renderer hierarchy are linked by a Renderer reference. Solid hollow-triangle arrows show inheritance; dashed hollow-triangle arrows show interface realization. Dashed arrows from Main show dependencies.

The open diamond denotes shared aggregation in UML: a renderer can be passed to more than one shape, as Main does. This is object composition in the Bridge sense of a has-a relationship. A filled diamond would incorrectly imply exclusive lifetime ownership. Constructors on concrete classes and the helpers Dimensions, DrawingFiles and DrawingStyle are omitted from this overview; the editable PlantUML source includes the constructor signatures.

Switching the same object

```
Shape circle = new Circle(40, vector);
circle.draw(); // creates circle.svg
circle.setRenderer(raster); // replace only the bridge reference
circle.draw(); // creates circle.png
circle.setRenderer(vector);
circle.draw(); // same Circle, switched back
```

Main also switches the same Square from vector to raster rendering. Both concrete renderers receive an output directory at construction. Shape subclasses never inspect a concrete renderer type or choose a file format. Drawing operations can throw IOException; Main reports the failure and exits with status 1.

3 Clean Code principles

The following annotated excerpts are taken from the submitted source. Each principle has a concrete effect on the separation, readability or reliability of the bridge.

1 Single responsibility and separation of concerns

```
// Circle.java: shape-side behavior
@Override
public void draw() throws IOException {
    renderer().drawCircle(radius);
}
```

Circle keeps and validates its geometry, then delegates. SVG markup is implemented in VectorRenderer; pixel painting and PNG encoding are implemented in RasterRenderer. A change in PNG encoding does not require editing Circle. The shared DrawingFiles helper owns directory creation and byte output, keeping that concern out of both shape classes.

2 Depend on an interface and inject collaborators

```
// Shape.java: the field is the interface type
private Renderer renderer;

protected Shape(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer, "renderer");
}
```

The abstraction accepts its implementation instead of constructing a VectorRenderer internally. This makes the implementation replaceable and lets the tests supply a RecordingRenderer. Only the composition code in Main needs to select concrete classes. The normal shape API exposes draw and setRenderer without exposing SVG or Java2D internals.

3 Open for extension through the implementor contract

```
// Renderer.java: every new renderer implements this contract
public interface Renderer {
    void drawCircle(int radius) throws IOException;
    void drawSquare(int side) throws IOException;
}
```

An additional renderer can implement these two operations and be passed to the existing constructors or setRenderer. Shape, Circle and Square require no edits for that extension. The test-only RecordingRenderer demonstrates this by implementing the interface outside the production renderer classes. A new primitive such as a triangle is a different change: the current interface would need another operation, implemented by every renderer. Extensibility is bounded by the contract, not unlimited.

3 Clean Code principles continued

4 Avoid duplicated policies and file handling

```
// DrawingFiles.java: reused by both concrete renderers
void write(String filename, byte[] content) throws IOException {
    Files.createDirectories(directory);
    Files.write(directory.resolve(filename), content);
}
```

Directory creation and writing encoded bytes have one implementation. Dimensions centralizes the 1-to-1000 size rule and DrawingStyle centralizes padding and color. The SVG and PNG algorithms remain separate because their responsibilities differ. Within VectorRenderer, writeSvg supplies the shared document wrapper; within RasterRenderer, writePng supplies the shared image setup, encoding and resource cleanup.

5 Meaningful names and small focused methods

```
// Square.java: names communicate geometry and the role of the collaborator
public Square(int side, Renderer renderer) {
    super(renderer);
    this.side = Dimensions.requireValid(side, "side");
}
```

Names such as radius, side, renderer, drawCircle and writePng express intent without a decoding step. Circle and Square have short constructors and single-purpose draw methods. The renderer methods build the shape-specific geometry and pass common work to a named helper. This keeps the example easy to inspect during the defense without introducing a framework or third-party dependencies.

6 Encapsulation and early validation

```
// Circle.java: geometry cannot change during renderer switching
private final int radius;

// Shape.java: reject an invalid replacement before assignment
public final void setRenderer(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer, "renderer");
}
```

The size fields are private and final. The renderer reference is private and replaceable only through the checked method. If null is supplied, assignment does not occur and the previous renderer remains usable. Invalid dimensions are rejected before drawing; the upper limit bounds image allocation and arithmetic. Concrete renderers apply the same validation policy when called directly. Graphics2D resources are released in a finally block, and I/O failures are propagated instead of being silently ignored.

4 Verification and conclusion

Observed verification results

Compilation passed with `--release 17 -Xlint:all -Werror`. The generated classes use major version 61, the Java 17 class-file format. All 14 automated tests passed under Oracle JDK 25.0.1 on Windows. The PowerShell and Git Bash launchers both passed. A separate execution on a JDK 17 runtime was not performed; the compiler restricted the source and referenced Java APIs to release 17.

Area	Checks performed
Bridge behavior	Radius and side stay unchanged; both shapes use both real renderers; switching back works.
Output files	SVG XML and geometry attributes; PNG decoding, dimensions, fill and background pixels.
Input and errors	Invalid and boundary sizes; null references; write failures; invalid replacement preserves the old renderer.
Portability	Arabic system locale cannot change SVG numbers; no GUI is needed; both launch scripts pass.

The demonstration creates `circle.svg`, `square.svg`, `circle.png` and `square.png`. With radius 40, side 80 and padding 12 on each side, every image is 104 by 104 logical units or pixels. Repeated draws replace the corresponding output file. Excess command-line arguments produce exit status 2; a file used as the output directory produces exit status 1. The test source and captured results are included in the repository.

Conclusion and trade-offs

Bridge separates geometry from rendering and supports runtime implementation replacement without rebuilding a shape. The benefit is clearest when another renderer is added: it can satisfy the existing interface without changing the shape classes. The cost is additional types and delegation compared with a single drawing class. The small interface is easy to understand but adding a genuinely new primitive requires changes on the renderer side. Fixed output names keep the demo simple; a drawing gallery would need an explicit naming or document model. The project is a single-threaded example, not a concurrent graphics engine.

GitHub repository

<https://github.com/maxiszero/assignment-3-bridge-pattern>

References

Assignment 3 Bridge Pattern Instructions, supplied course handout.

[Refactoring.Guru - Bridge](#) (assigned conceptual reading).

[Oracle Java SE 17 - Graphics2D](#); [Oracle Java SE 17 - ImageIO](#) (API documentation).